

Theory Reference — Ecosystem Simulation

Yermek Aubayev (551098)

2026-09-14

Companion to `defense_script.md` (the rehearsal / Q&A / live-coding file). This file is the lookup reference: one concept per entry, a definition, where it lives in the actual code, and the one sentence worth saying out loud. Every line number below was checked against the current source on 2026-09-14, not copied from an older draft — if you edit the code after this date, re-check before relying on a line number in front of the professor.

Terminology matches `docs/report/final_report.md` exactly on purpose. Do not improvise a different word for the same idea in the defense (e.g. don't say “event queue” in one breath and “task list” in the next) — the report and the spoken defense need to sound like the same person wrote/understood both.

Part A — OOP Principles

A.1 Encapsulation

Definition. Bundling an object's data with the operations that act on it, and controlling how that data can be reached from outside — the object's own methods are the only door in.

Code. [World.java:85](#) — `getEntities()` returns `new ArrayList<>(this.entities)`, a defensive copy, not the live list.

Say this. “A `private` field only stops you from *replacing* World's reference. Without the defensive copy, a caller could still get the live list through the getter and mutate it — add or remove entities behind World's back. The copy is what actually closes that door.”

⚠ **Previously challenged.** The professor flagged that an earlier version of this section mixed encapsulation with information hiding. Keep them visibly separate — see A.2.

A.2 Information Hiding

Definition. Choosing, for every attribute, exactly the visibility, scope, and mutability it needs — so no code outside the declaring class can come to depend on details that might change later. This project applies a 3-step method to every field: (1) private? (2) instance- or class-scoped? (3) constant or variable?

Code. [Entity.java:24](#) (`private final int id`) and [Entity.java:29](#) (`private static int nextId = 1`).

Attribute	Private?	Scope	Constant/variable
<code>Entity.id</code>	Yes	Instance	Constant (<code>final</code> , set once in the constructor)
<code>Entity.nextId</code>	Yes	Class (<code>static</code>)	Variable (increments every <code>new Entity(...)</code>)
<code>Organism.energy</code>	Yes	Instance	Variable (<code>gainEnergy</code> / <code>consumeEnergy</code>)

Say this. “Every field in the `Entity` → `Organism` → `Animal` → `Predator/Herbivore` chain is `private` , including fields declared in the abstract base classes — not `protected` . A subclass reaches inherited state only through the accessor, the same as any outside caller would.”

⚠ **Previously challenged — know this cold.** The professor caught real `protected` fields despite accessor-based claims (`Entity.id/x/y/alive/...` , `Organism.energy/age/maxEnergy` , `Animal.speed/visionRange` , `Predator.attackPower` , `Herbivore.defensePower`), and caught `Predator.hunt()` reading the inherited `visionRange` field directly instead of calling `getVisionRange()` . Both are fixed now — verified: every field above reads `private` in the current source (checked directly, not from the report). **Encapsulation vs. information hiding, in one sentence:** encapsulation is about *access control on one object’s own state* (the method’s contract); information hiding is about *whether a hierarchy’s internal representation is free to change* (the field’s declaration). Don’t restate one as the other.

A.3 Inheritance

Definition. A subclass reuses and extends a superclass’s state and behavior, expressing a genuine “is-a” relationship.

Code. `Organism.java:15` extends `Entity` , `Animal.java:16` extends `Organism` implements `Movable` , `Herbivore.java:9` extends `Animal` implements `Reproducible` .

Say this. “ `Plant` sits directly under `Organism` , not under `Animal` , even though it’s obviously a living thing. That was a deliberate check: if `Plant` inherited from `Animal` implements `Movable` , code holding an `Animal` reference could legally call `move()` on something that structurally cannot move — a Liskov Substitution violation. Keeping `Movable` a separate interface (A.7) is what makes that tree honest.”

A.4 Abstraction

Definition. Representing only the essential features an object must have, deferring *how* to concrete subclasses.

Code. `Entity.java:56` `public abstract void update();` and `SimulationEvent.execute(SchedulingContext) / getDescription()` .

Say this. “The base class can’t know the concrete behavior in advance — a `Plant` grows, a `Predator` hunts, a `DeathEvent` records a death regardless of cause. One abstract method per concern is what lets the engine stay completely ignorant of concrete types (A.5.1).”

A.5 Polymorphism — four distinct forms

A.5.1 Inclusion (runtime / dynamic dispatch)

Code. [SimulationEngine.java:132](#) — `event.execute(this)` , dispatched at runtime to whichever of the six event subclasses was popped.

Say this. “The engine never asks ‘what kind of event is this?’ It calls `execute()` through the base `SimulationEvent` type and the JVM’s dynamic dispatch selects the override. Same mechanism one level down: `EntityActivityEvent` calls `entity.update()` through the base `Entity` type.”

Honest non-example, worth stating unprompted. The rule AST’s sealed nodes (`ConditionNode` , `NumericNode` , `RuleAction` — see Part C) are evaluated in `Evaluator` through an exhaustive `switch` ([Evaluator.java:37](#)), **not** virtual dispatch — there is no `evaluate()` method per node. That’s a deliberate, modern alternative (compiler-checked exhaustiveness over a closed node set), not an instance of inclusion polymorphism. Being able to draw this line unprompted is exactly the kind of precision that reads well in an oral exam.

A.5.2 Overloading (compile-time)

Code. [Statistics.java:85](#) `recordBirth(String)` and [Statistics.java:100](#) `recordBirth(String, int)`; [RuleParseException.java](#) has four constructors (lines 31/43/58/70), all named `RuleParseException` but distinguished by parameter list.

Say this. “The compiler picks the matching signature at **compile time**, from the argument types at the call site — the opposite resolution moment from `update()` / `execute()` in A.5.1, which resolve at **runtime** from the object’s actual class. That compile-time-vs-runtime distinction is the single most common follow-up question here.”

A.5.3 Parametric (generics)

Code. [EventQueue.java:28](#) — `class EventQueue<T extends SimulationEvent>` .

Say this. “The bound does real compile-time work: the compiler rejects enqueueing anything that isn’t a `SimulationEvent` , while `EventQueue` itself never needs to know which concrete subtype it holds. Only `EventQueue<SimulationEvent>` is actually instantiated in this project, but the bound isn’t decorative — a future, narrower event queue would work with this class completely unmodified.”

A.5.4 Coercion

Code. [Predator.java:170](#) — `(double) this.attackPower / (this.attackPower + herbivore.getDefensePower())` .

Say this. “Both operands start as `int` . The explicit cast on the left operand forces Java’s binary numeric promotion rule to widen the *other*, uncast `int` operand to `double` too, before the division runs — that implicit widening of the second operand is the coercion. This is deliberately **not** illustrated with `Integer.parseInt(...)` — that’s an explicit method call performing a conversion, not a language-level coercion. Conflating the two is a labeling mistake worth avoiding out loud.”

A.6 Composition

Definition. A “has-a” relationship: an object holds references to other, independently constructed objects and uses their public behavior.

Code. [SimulationEngine.java:36-42](#) — six `final` fields (`World`, `RuleVocabulary`, `RuleEngine`, `Statistics`, `EntityFactory`, `EventQueue<SimulationEvent>`), all constructed inside `SimulationEngine`’s own constructor.

Say this. “None of these composed objects *is a kind of* `SimulationEngine`, and none meaningfully outlives one running simulation — so composition, not inheritance, is the structurally correct relationship. It also keeps the engine’s internals encapsulated: nothing outside can reach `RuleEngine` except through the narrow `SchedulingContext` seam the engine itself implements.”

A.7 Subtyping / Multityping (interfaces)

Definition. One object simultaneously satisfying several independent types.

Code. `Predator` is an `Animal` (inheritance), a `Movable` (inherited transitively through `Animal`), and a `Reproducible` (declared directly) — three types, one object. Proof lives in [Main.java:67](#) (`demonstrateMultityping()`, called from `main()` at [Main.java:23](#)) and in [RuleVocabulary.java:244-246](#), where `move` / `reproduce` commands are registered against the **interface**, not a list of concrete classes.

Say this. “`Main.demonstrateMultityping()` builds exactly one `Predator` and assigns that same reference to five variables of five different declared types — `Entity`, `Organism`, `Animal`, `Movable`, `Reproducible` — then prints `entityRef == animalRef`, which is `true`. That’s the proof: reference equality compares the underlying object, not the compile-time type of the variable holding it.”

⚠ **Previously challenged.** The professor asked for exactly this client-program proof; it didn’t exist before. `Main.demonstrateMultityping()` was written specifically in response and runs at the start of every program execution, before the GUI opens — be ready to actually run the program and show the console output live if asked.

A.8 Exception Handling

Definition. Code that can fail signals it through a well-defined exception type, and a caller that expects the failure **catches it and takes a real recovery action** — not just defining an exception class and letting it propagate, and not just catching-and-printing.

Code. Checked: [RuleParseException.java:13](#) extends `Exception`. Unchecked: [SimulationException.java:32](#) extends `RuntimeException`. Real catch + recovery #1: [RuleEditorPanel.java:119-127](#) `doReload()`. Real catch + recovery #2: [MainWindow.java:61-69](#) and the matching block at line 185.

Say this. “`RuleParseException` is checked because callers are *expected* to catch it and recover — a malformed rule is ordinary user input, not a bug. `SimulationException` is unchecked because its causes (e.g. negative world dimensions) indicate a programming error, not something to routinely expect and recover from at runtime. `RuleEditorPanel.doReload()` is genuine handling, not decoration: because `RuleEngine.loadRules()` parses into a staging list and only replaces the active

rules on full success, catching `RuleParseException` here leaves the engine's active rules **exactly as they were** — verified by `RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules`, not just asserted in prose.”

⚠ **Previously challenged.** The professor said the original exception section only showed constructor definitions, never an actual `catch` block, and separately, `MainWindow`'s original catch only wrote to `System.err` — invisible in a packaged Swing app, arguably not real handling since nothing about the program's visible behavior changed. Both `MainWindow` catch sites now log to the on-screen `EventLogPanel` instead. If asked “what does handling actually require, beyond a try/catch existing” — say: it must (1) stop the exception from crashing the program, (2) leave the program in one specific, well-defined state, (3) tell the user what happened. A catch block that only prints to a console nobody sees satisfies none of the second or third condition.

A.9 Extensibility (Open/Closed Principle)

Definition. New features can be added without modifying existing, already-tested classes.

Code. [RuleVocabulary.java:75-85](#) — `registerReadable / registerWritable / registerCommand`, the one seam a new attribute or command goes through.

Say this precisely — don't overstate it. “Composing *new rule logic* from attributes/commands already registered needs zero Java changes. Exposing a genuinely *new* attribute or command needs exactly one `registerX(...)` call in `RuleVocabulary` and a recompile — nothing in `Lexer`, `Parser`, `Rule`, or `RuleEngine` changes. That boundary is stated openly in the report; don't let a question push you into claiming the DSL is more open than that.” See [defense_script.md §5](#) for the sharper nuance on species extensibility specifically — the report's “the engine needs no changes” claim needs one caveat when you actually add a species live.

A.10 Modularity

Definition. Software built as small independent pieces, each owning one concern; in OOP the module *is* the class.

Code. Package structure: `entities/`, `events/`, `rules/`, `simulation/`, `statistics/`, `gui/` — `entities` never imports `gui`; `rules` knows `entities` only through `Movable / Reproducible / Entity`, never a concrete species.

Say this — and volunteer the honest gap, don't wait to be caught. “It's not perfectly modular everywhere, and that's stated in the report rather than hidden: the concrete species list (`Predator / Herbivore / Plant`) is spelled out independently in two places — `Statistics.adjustSpeciesCounter` ([Statistics.java:130](#)) and `EntityFactory.createOffspringNear`'s `instanceof` chain ([EntityFactory.java:78-87](#)). Removing that would mean adding a species-registry abstraction solely to delete one small `instanceof` chain — a deliberate tradeoff, not an oversight, trading a generic mechanism for something a new reader can actually follow.”

A.11 Hierarchy

Definition. Organizing related concepts by specialization — not just that two things are related, but which is more general and which is more specific, and how many levels the domain actually needs.

Code. The four-level tree: `Entity → Organism → Animal → {Predator, Herbivore}`, with `Plant` branching off `Organism` directly.

Say this. “This isn’t just documentation — `RuleVocabulary.requireReadable / requireWritable / requireCommand` validate a rule by walking this exact tree with `Class.isAssignableFrom`, so a rule targeting `Organism` is automatically valid for everything below it without `RuleVocabulary` ever naming `Predator` or `Herbivore`. Get the shape wrong — e.g. `Plant` under `Animal` — and an invalid rule like `Plant | move` would pass validation, because the vocabulary trusts the hierarchy to tell the truth about what a type actually is.”

A.12 Reuse

Definition. Applying existing, already-debugged code to a new need, instead of rewriting it. Two mechanisms: inheritance (reusing a superclass’s *internal* state through its own accessors) and composition (reusing another object’s *public* behavior by holding a reference to it).

Code. Inheritance-reuse: `Predator / Herbivore` both reuse `Organism`’s energy/age management and `Animal`’s movement state. Composition-reuse: `EntityFactory` — before it existed, species-default construction logic was duplicated once for the initial population and again inside each species’ own `reproduce()`; a config change in one place silently failed to reach the other.

Part B — Discrete-Event Simulation Theory

General definition (textbook, not project-specific). A discrete-event simulation models a system as a sequence of instantaneous events, each of which changes system state at a specific point in simulated time. Time advances only by jumping to the next scheduled event — never by a fixed Δt — because between events nothing changes, so stepping through the gaps would be wasted computation. This is the opposite of **time-slicing / fixed-timestep simulation**, which advances a constant Δt every iteration and re-evaluates every entity whether or not anything about it changed.

Core vocabulary, and where each concept lives in this project:

DES concept	This project
Simulation clock	<code>SimulationEngine.clock</code> (private <code>int</code>)
Future-event list (FEL)	<code>EventQueue<SimulationEvent></code> , backed by <code>java.util.PriorityQueue</code> — EventQueue.java:28
Event	<code>SimulationEvent</code> and its six subclasses — SimulationEvent.java:36
Event’s own timestamp	<code>scheduledTime</code> , set once at construction — SimulationEvent.java:41
State variables	<code>Entity / World / Environment / Statistics</code> fields — mutated only inside some event’s <code>execute()</code>
The driving loop	<code>SimulationEngine.advanceTo(int)</code> — SimulationEngine.java:128

The one sentence that proves this is genuinely discrete-event, not a fixed loop wearing the

name: `advanceTo()` pops the single earliest-scheduled event and sets `clock = event.getScheduledTime()` — **it adopts the popped event's own time**, rather than incrementing the clock by a fixed step and then polling every entity. The clock never advances on the engine's own initiative; it only ever adopts a timestamp that already existed on an event.

Recurring processes, the standard DES idiom. No entity is ever polled by an external loop. Every living entity has exactly one pending `EntityActivityEvent` at a time; executing it performs the entity's behavior *and reschedules its own successor* — that self-rescheduling is what makes it a recurring process in DES terms, and it's also why a dead entity's chain simply stops (nothing reschedules a successor for it).

One owner per mutation. Every state change has exactly one event class responsible for it — deliberately, so “why did X change” always has exactly one place to look:

Mutation	Owner
Aging, energy consumption, growth, movement decision, rule evaluation	<code>EntityActivityEvent.execute()</code>
Predator-side kill effects	<code>PredationEvent.execute()</code>
Offspring creation, birth stat, first activity event	<code>ReproductionEvent.execute()</code>
<code>alive</code> flag, world removal, death stat, for every cause	<code>DeathEvent.execute()</code> — the only call site of <code>Entity.die()</code>
Position-change record	<code>MovementEvent</code> — observational only, the move already happened inside <code>EntityActivityEvent</code>

Lazy invalidation. An entity can die between when an event referencing it was scheduled and when that event actually runs. `PriorityQueue` has no efficient arbitrary-removal operation, so nothing actively hunts the FEL to cancel stale entries — instead, every event re-checks `isAlive()` / `isPendingRemoval()` at the top of its own `execute()` and silently no-ops if stale. `Entity.pendingRemoval` ([Entity.java:38](#)) is the guard, set synchronously the instant removal is *requested* ([Entity.java:165](#) `scheduleRemoval`), before the `DeathEvent` that will actually flip `alive` even runs.

Deterministic tie-breaking. `java.util.PriorityQueue` does not guarantee FIFO order among elements that compare equal. `SimulationEvent.compareTo` ([SimulationEvent.java:87](#)) breaks ties on a strictly monotonic `sequenceNumber` (an `AtomicLong`, assigned at construction) — two events scheduled for the same tick are always processed in creation order.

Known, stated limitation — say this before being asked. “Time is integer-tick-valued, and in this particular ecosystem model, every entity needs an activity roughly every tick, so the *visible cadence* resembles a fixed timestep. The load-bearing, architectural property isn't that inter-event gaps happen to be large — it's that the clock is FEL-driven (it always adopts a popped event's own time) and that state only ever changes inside `execute()`. A model where gaps *were* large (say, a plant that only acts every 50 ticks) would immediately look different from a polling loop; this one doesn't have to, to still be genuinely event-driven.”

Part C — Rule DSL / Compiler Theory

Pipeline, in order: free text → **Lexer** (tokenizer) → **Parser** (recursive-descent, hand-written, no ANTLR or other generator) → **AST** (sealed interfaces + records) → static **validation** against `RuleVocabulary` → **Evaluator** (runtime, once per entity per tick).

Grammar (EBNF-style, precedence low → high):

```
rule      ::= name "|" targetType "|" condition "|" actionList
condition ::= orExpr
orExpr    ::= andExpr ( "OR" andExpr )*
andExpr   ::= unary ( "AND" unary )*
unary     ::= "NOT" unary | comparison | "(" orExpr ")"
comparison ::= numExpr compareOp numExpr
numExpr   ::= term ( "+" | "-" ) term
term      ::= factor ( "*" | "/" ) factor
factor    ::= NUMBER | IDENTIFIER | "-" factor | "(" numExpr ")"
actionList ::= action ( "," action )*
action    ::= IDENTIFIER ( mutationOp numExpr )?
mutationOp ::= "=" | "+=" | "-=" | "*=" | "/="
```

Precedence, low to high: `OR` < `AND` < `NOT` < `comparison` < `+` `-` < `*` `/`. This is the classic shape of a hand-written recursive-descent expression grammar — each grammar rule becomes one parser method, and precedence is encoded by *which method calls which* (`orExpr` calls `andExpr` calls `unary` ...), not by a precedence table.

Lexer. Converts raw characters into a flat token stream (`TokenType` + lexeme + position). Knows nothing about grammar structure — only character classes (digit, letter, operator symbol) and keyword recognition (`AND`, `OR`, `NOT`).

Parser. Recursive-descent: each nonterminal in the grammar above is one method (`parseOrExpr()`, `parseAndExpr()`, ...), and each method calls the next-higher-precedence method for its operands, which is how the precedence climbing happens structurally rather than via a table.

AST — sealed interfaces + records (Java 21), not a classic visitor. `ConditionNode` (`ConditionNode.java:11`) is sealed, permitting only `And`, `Or`, `Not`, `Compare` — all records. Same shape for `NumericNode` (`NumericNode.java:18`: `Literal`, `Reference`, `BinaryOp`) and `RuleAction` (`RuleAction.java:12`: `Mutation`, `Command`). `Evaluator` consumes these with an exhaustive pattern-matching `switch` (`Evaluator.java:37`, `:53`, `:64`, `:98`) — the compiler refuses to compile if a case is missing, which is what “closed and exhaustively checked” buys you that a classic OO visitor pattern (double dispatch through an `accept()` / `visit()` pair on every node) would need a separate interface and a full visitor implementation to get, and even then without compile-time exhaustiveness checking pre-Java-21.

Static vs. dynamic (runtime) validation — one consistent principle. Anything detectable without live simulation state — an unknown attribute, a target/command type mismatch, a literal-zero divisor — is a hard `RuleParseException` at **load time** (`RuleVocabulary.java:101-144`, `requireReadable` / `requireWritable` / `requireCommand`). Anything only discoverable from *changing* runtime state — a stale scheduled reference, a divisor that’s zero only because some other rule just made it zero — is a graceful, silent no-op inside `Evaluator`, never a crash.

Level-triggered evaluation. A rule fires again on every tick its condition still holds — there is no per-rule “already fired” flag to track, which is simpler than edge-triggered (“fire once when it becomes true”) but means a rule author must build hysteresis into the rule itself if they want edge-triggered behavior (e.g. combine the condition with a cooldown attribute).

Numeric safety. All rule arithmetic is `double`. A runtime division by a dynamic zero yields `NaN` / `Infinity`; `Evaluator` treats either as a false condition or a skipped mutation — never as a value that propagates into an entity’s actual state.

The honest compile-time boundary — state this precisely. Composing *new rule logic* from attributes/operators/commands already registered in `RuleVocabulary` needs zero Java changes or recompilation. Exposing a genuinely *new* attribute or command needs one `registerReadable` / `registerWritable` / `registerCommand` call and a recompile. Both halves of that sentence are true; say both, not just the flattering half.

Part D — Design Principles Referenced by Name in the Report

- **Liskov Substitution Principle.** A subtype must be usable anywhere its supertype is expected without surprising behavior. Cited for *why* `Plant` is not under `Animal` (A.3) and as `Movable`’s own contract (“calling `move()` must result in a valid position change”).
- **Interface Segregation Principle.** No class should be forced to implement a method it has no sensible body for. Cited for *why* `Movable` and `Reproducible` are two interfaces, not one (A.7) — `Plant` is `Reproducible` but not `Movable`.
- **Open/Closed Principle.** Open for extension, closed for modification. This is the formal name for A.9 (Extensibility).

Part E — Compressed Glossary (last five minutes before walking in)

- **Encapsulation** = access control on *this object*’s state (method contracts). **Information hiding** = whether a *hierarchy*’s representation can change freely later (field declarations, 3-step method). Different axes — don’t conflate.
- **Inclusion polymorphism** resolves at **runtime**, from the object’s actual class (`update()`, `execute()`). **Overloading** resolves at **compile time**, from argument types at the call site. **Parametric** = generics (`EventQueue<T extends SimulationEvent>`). **Coercion** = implicit widening triggered by an explicit cast on a sibling operand.
- **FEL-driven** = clock adopts a popped event’s own timestamp. **Fixed-timestep** = clock increments by a constant, then polls everything. This project is the former — `advanceTo()`.
- **Checked exception** (`RuleParseException extends Exception`) = caller is *expected* to catch and recover — malformed user input. **Unchecked** (`SimulationException extends RuntimeException`) = a programming-contract violation, not routinely recoverable.
- **Sealed interface + record + exhaustive switch** = a closed AST evaluated without virtual dispatch, with the compiler enforcing every case is handled — the deliberate alternative to a classic visitor pattern here.

- **Level-triggered** = a rule re-fires every tick its condition holds (no “already fired” memory). Not edge-triggered.